Agenda:

1. Count set bits
2. Single Number
3. Single Element 2 (Every element 3 times and one unique element)
4. Single Number 3 (Every element 2 times and 2 unique elements)

Q1: Given an integer **N**, count the total number of **SET** bits (= **1**) in N.

Example: N = 12 Ans = 2 12 = (1 1 0 0) (3 2 1 0)

N = 45 Ans = 4 45 = (1 0 1 1 0 1)

int → 32 bits

```
ans = 0
for i → 0 to 31 {
    if ( N & (1<<i) > 0 )
        ans ++
}
return ans
```

long 63

ans += (N >> i) & 1

TC = O(32) = O(1) SC = O(1)

N = 45 ... 1 0 1 1 0 1 → discard (1 0)

N & 1

N >> 1: 1 0 1 1 0

1 0 1 1

```
ans = 0
while (N > 0) {
    ans += (N & 1)
    N = N >> 1        // N/2
}
return ans
```

N → integer

$\log_2(N) <= 32$

TC = $O(\log_2(N))$ SC = O(1)

Q2: Given an integer array **A[ ]** where every number occurs **twice** except for one number which occurs just **once**. Find that number.

$A[i] <= 10^9$

Example: A = [4, 5, 5, 4, 1, 6, 6] Ans = 1

A = [7, 5, 5, 1, 7, 6, 1, 6, 4] Ans = 4

$4 \wedge 4 \wedge 5 \wedge 5 \wedge 6 \wedge 6 \wedge 1$

$= \underline{0 \wedge 0} \wedge \underline{0 \wedge 1} = 0 \wedge 1 = \underline{1}$

Sol 1 → Find frequency of every number.

Element with freq = 1 is answer.

TC = $\underline{O(N)}$ SC = $\underline{O(N)}$ → frequency array

TC = $\underline{O(N^2)}$ SC = $\underline{O(1)}$ → bruteforce

Sol 2 → Ans = $\underline{\forall i \wedge A[i]}$ → TC = $\underline{O(N)}$ SC = $\underline{O(1)}$

	3	2	1	0
5 →	0	1	0	1
5 →	0	1	0	1
4 →	0	1	0	0
11 →	1	0	1	1
11 →	1	0	1	1
9 →	1	0	0	1
4 →	0	1	0	0
# 1's →	3	4	2	5

	0	1	2	3	4	5	6
A[ ] →	5	5	4	11	11	9	4

1's at any position

odd (2x+1) ⇒ 1 in ans

even (2x) ⇒ 0 in ans

$1 \wedge 1 = 0$ $0 \wedge 0 = 0$

$1 \wedge 0 = 1$ $0 \wedge 1 = 1$

1 0 0 1 → 9

```
ans = 0;

for(i -> 0 to 31) { // go to every bit one by one
    cnt = 0;

    for(j -> 0 to (A.length - 1)) { // iterate on array

       // check if ith bit is set
       if((A[j] & (1<<i)) == 1)    cnt++;
    }

    if((cnt & 1) == 1) // If the count is odd
        ans = ans | 1 << i; // set ith bit in ans
}

print(ans);
```

Q3: Given an integer array **A[ ]** where every number occurs **three** times except for one number which occurs just **once**. Find that number.

TC = O(N) SC = O(1)

Example: A = [4, 5, 5, 4, 1, 4, 5] Ans = 1

A = [7, 6, 7, 1, 7, 6, 1, 6, 4, 1] Ans = 4

	0	1	2	3	4	5	6	7	8	9
A[ ] →	5	5	4	11	11	9	11	5	4	4

	3	2	1	0
5 →	0	1	0	1
5 →	0	1	0	1
4 →	0	1	0	0
11 →	1	0	1	1
11 →	1	0	1	1
9 →	1	0	0	1
11 →	1	0	1	1
5 →	0	1	0	1
4 →	0	1	0	0
4 →	0	1	0	0
#1's	4	6	3	7

#1's at any position

multiple of 3 (3x) ⇒ 0 in ans

not a multiple of 3 (3x+1) ⇒ 1 in ans

```
ans = 0
for i → 0 to 31 {        // bit position
    cnt = 0
    for j → 0 to (N-1) {
        cnt += (A[j] >> i) & 1
    }
    if (cnt % 3 == 1)
        ans = ans | (1<<i)        // set i^th bit
}
return ans
```

K

cnt = # 1's at i^{th} position

$TC = O(32 * N) = \underline{O(N)}$

$SC = \underline{O(1)}$

Q4: Given an integer array **A[ ]** where every number occurs **twice** except for **two** numbers which occurs just **once**. Find those two numbers.

TC = O(N) SC = O(1)

Example: A = [4, 5, 4, 1, 6, 6, 5, 2] Ans = {1, 2}

Sol 1 → Find frequency of every number.

Elements with freq = 1 are answers.

TC = O(N) SC = O(N) → frequency array

TC = $O(N^2)$ SC = O(1) → bruteforce

Sol 2 →

	0	1	2	3	4	5	6	7
A =	4	5	4	1	6	6	5	2

Step 1 → $\forall i, \wedge A[i]$ $4 \wedge 5 \wedge 4 \wedge 1 \wedge 6 \wedge 6 \wedge 5 \wedge 2$

$= 1 \wedge 2 = 3$ → $1 \wedge 2$, $11 \wedge 8$, $4 \wedge 7$, ⋮

$x \wedge y > 0$

$x != y$

atleast 1 bit is set.

which is set in x or y

but not both.

$x = 5$

$y = 6$

$$\begin{array}{r} 101 \\ \wedge\ 110 \\ \hline 011 \end{array}$$

Step 2 → Find any set bit in xor of all

6 → 110 (2 1 0) Ans → 1 or 2

$$\begin{array}{cccccccc} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ \end{array}$$

$A = [4 \quad 5 \quad 4 \quad 1 \quad 6 \quad 6 \quad 5 \quad 2]$

$$\begin{array}{ccc} 2 & 1 & 0 \\ 0 & 1 & 1 \end{array}$$

xor = $\underline{3}$

$b = 0$ // position of set bit in xor

<u>Step 3</u> → Divide A[] 2 parts

1 → All numbers where b^{th} bit is 0

2 → All numbers where b^{th} bit is 1

1 → {4 4 6 6 2}

2 → {5 1 5}

<u>Step 4</u> → Ans 1 = xor of all in set 1

Ans 2 = xor of all in set 2

$A = [\overset{\rightarrow}{2} \quad 8 \quad 4 \quad 4 \quad 5 \quad 5 \quad 6 \quad 6]$

$$\begin{array}{cccc} 3 & 2 & 1 & 0 \\ 1 & 0 & 1 & 0 \end{array}$$

xor = $2 \wedge 8 = \underline{10}$

$b = \underline{1}$

b^{th} bit 0 → {8 4 4 5 5} → $\underline{8}$

b^{th} bit 1 → {2 6 6} → $\underline{2}$

$$\begin{array}{cccccc} & & & 3 & 2 & 1 & 0 \\ 2 & \rightarrow & & 0 & 0 & 1 & 0 \\ 8 & \rightarrow & & 1 & 0 & 0 & 0 \\ 4 & \rightarrow & & 0 & 1 & 0 & 0 \\ 5 & \rightarrow & & 0 & 1 & 0 & 1 \\ 6 & \rightarrow & & 0 & 1 & 1 & 0 \end{array}$$

Step- 1 : Take XOR of all the elements.

```
xor = 0
for i → 0 to (N-1) {
    xor = xor ^ A[i]
}
```

Step- 2 : Find any set bit position in val.

```
b = -1
for i → 0 to 31 {
    if ((xor & (1<<i)) > 0) {
        b = i        break
    }
}
```

Step- 3 : Split the array on the basis of pos'th bit.

```
ans1 = 0          ans2 = 0
for i → 0 to (N-1) {
    if ((A[i] & (1<<b)) == 0)   // b^th bit is 0
        ans1 = ans1 ^ A[i]
    else                        // b^th bit is 1
        ans2 = ans2 ^ A[i]
}
return {ans1, ans2}
```

$TC = O(N + 32 + N) = \underline{O(N)}$ $SC = \underline{O(1)}$

```
    xorAll = 0;

    // XOR of all numbers in the array
    for (i -> 0 to N - 1) {
        xorAll ^= A[i];
    }

    // Find the rightmost set bit position
    // Note: Any other bit can be used as well
    declare pos

    for(pos = 0; pos < 32; pos++)
    {
        if(checkbit(xorAll,pos))
            break;
    }

    num1 = 0;
    num2 = 0;

    // Divide the array into two groups based on the rightmost set bit
    for (i -> 0 to N - 1) {
        if (checkbit(A[i], pos)) {
            num1 ^= A[i];
        } else {
            num2 ^= A[i];
        }
    }

   print(num1);
   print(num2);
```

What will be the outpout for following code snippet?

```
int[] arr = {5, 5, 5, 9};
int ans = 0;

for(int i = 0; i < 32; i++){
    int cnt = 0;

    for(int x : arr){
        if((x & (1 << i)) != 0)
            cnt++;
    }

    if(cnt % 3 == 1)
        ans |= (1 << i);
}

System.out.println(ans);
```

→ 9